

Kawayan AI — Technical Manual & Capstone System Guide

1. Introduction

Purpose of the System

Kawayan AI is an intelligent, localized social media content automation and management platform tailored for Filipino Small and Medium Enterprises (SMEs). The system bridges the digital gap for local businesses by providing an automated, Taglish-optimized content creation calendar, hybrid cloud/local AI generation, transaction-safe wallet payment flows, and integrated help desk channels (including real-time VoIP support).

Project Overview

Filipino SMEs often face barriers to digital transformation due to budget constraints, lack of dedicated marketing personnel, and language mismatches in corporate AI models. Kawayan AI addresses these problems directly. It acts as an autonomous digital marketing manager that can: 1. Analyze a brand's unique profile (voice, audience, theme). 2. Fetch real-time domestic search trends. 3. Automatically generate a localized, culturally aligned social media content plan. 4. Render ready-to-publish drafts complete with localized Taglish captions and visual prompts.

Objectives

- **Promote Digital Inclusivity:** Empower local micro-businesses (cafes, bakeries, gadget shops) to establish consistent online brand presences.
- **Culturally Grounded Generation:** Provide AI text generation optimized for "Taglish" (mixed Tagalog/English), mirroring the organic communication style of modern Philippine social networks.
- **System Resilience:** Architect a hybrid AI engine utilizing low-latency cloud models (Google Gemini) that seamlessly falls back to a locally hosted, offline-capable model (Ollama / Qwen2.5) during connectivity losses.
- **Prepaid Self-Service Wallet:** Implement an invoice-based prepaid billing engine integrated with real payment gateways (Xendit) supporting domestic payment methods (GCash, Maya, cards).
- **Direct Customer Assistance:** Establish a direct WebRTC-based support calling mechanism connecting SMEs to support agents via high-quality audio/video streaming.

Scope

- **In-Scope:** User authentication and session management; Brand profile mapping; Automated monthly batch content planning; Taglish caption generation and prompt-to-image rendering (via Pollinations AI); Prepaid ledger wallet top-ups and subscription adjustments; Real-time chat tickets and WebRTC signaling server; Admin panels with audit trail streams and payment approvals; Browser extension statistics syncing.
- **Out-of-Scope:** Native publication execution (direct API posting to Facebook/TikTok remains manual or simulated through a browser extension helper due to platform API sandbox restrictions); Automated credit card recurring billing.

Intended Users

1. **SME Business Owners (Standard Users):** Non-technical users seeking cost-effective social media automation and billing controls.
2. **Support Agents:** Customer success operators responding to support tickets, reviewing business contexts, and launching WebRTC support calls.
3. **System Administrators:** IT managers monitoring system growth, executing security logs audits, managing users, and verifying/approving ledger balances.

2. System Overview

High-Level Explanation

Kawayan AI is designed around a decoupled, three-tier architecture: 1. **Presentation Layer (React Single Page Application)**: Renders a modern interface featuring an interactive editorial calendar grid (Schedule-X), billing logs, and support widgets. 2. **Gateway Layer (Node.js/Express API & WebSocket Server)**: Acts as a security boundary, orchestrating authentication, database transactions, image and audio streams, and proxying external API endpoints. 3. **Data Layer (SQLite & local Storage)**: Delivers persistence. The system utilizes a unified database abstraction layer (UniversalDatabaseService) to transparently switch modes depending on execution contexts.

Main Features

- **Interactive Editorial Calendar**: Interactive view displaying posts, drafts, and ideas in monthly and weekly formats.
- **Prepaid Billing & Add-Ons**: Invoicing system built with GCash/Maya integrations, with add-on mechanisms bypassing monthly post caps.
- **Growth Insights**: Dashboard tracing organic metrics (views, followers, ROI values) updated via manual tracking or extension scrapes.
- **Integrated Help Desk & VoIP**: Real-time customer support console complete with WebRTC media negotiation signaling.
- **Business Verification Portal**: Upload portal for regulatory compliance, whitelisting documents (DTI/SEC registration, business permits).

Problems the System Solves

- **High Creative Overhead**: Eliminates the cost of marketing copywriters.
- **Platform Lock-In & Cloud Downtime**: Solves connectivity failures via local offline fallbacks.
- **Manual Billing Hurdles**: Solves manual payment tracking through payment gateways (Xendit) and admin validation flows.

3. Relationship to the Capstone Paper

```
graph TD
  subgraph Paper [Capstone Research Paper Chapters]
    Ch1[Chapter 1: Research Objectives]
    Ch3[Chapter 3: Methodology]
    Ch4[Chapter 4: System Requirements & Design]
    Ch5[Chapter 5: Implementation & Testing]
    Ch6[Chapter 6: Evaluation & Expected Outcomes]
  end

  subgraph Implementation [Implemented System Features]
    Obj[Taglish Localized Text Engine]
    Method[Hybrid Cloud/Local AI Fallback Engine]
    ReqDesign[Interactive Editorial Calendar & Database Schema]
    ImplTest[REST API Gateway & System Tests]
    EvalOut[Prepaid Wallet, VoIP Support & Social Analytics]
  end

  Ch1 --> Obj
  Ch3 --> Method
  Ch4 --> ReqDesign
  Ch5 --> ImplTest
  Ch6 --> EvalOut
```

Research Alignment Map

Capstone Section	Core Research Objective / Methodology	Implemented System Capability	Rationale & Justification
Chapter 1: Objectives	To create localized, Taglish-fluent marketing copy for Philippine SMEs.	Taglish Generation Prompts: Configured system prompt directives utilizing Gemini Qwen2.5.	Translates high-level business ideas into warm, colloquial Filipino & dialects suitable for target demographics.

Capstone Section	Core Research Objective / Methodology	Implemented System Capability	Rationale & Justification
Chapter 3: Methodology	Hybrid network resilience using local client edge nodes during offline intervals.	Hybrid AI Service (geminiService): Automatic transition to Ollama/Unsloth endpoint on cloud failure.	Validates the architectural thesis that software can remain operational without active cloud bandwidth.
Chapter 4: Requirements	A centralized workspace for tracking scheduled content and campaign assets.	Interactive Editorial Calendar: Full CRUD interface integrated with Schedule-X.	Provides users with a single point of reference to manage draft, scheduled, and published assets.
Chapter 4: Design	Relational data integrity for billing, auditing, and multi-tenant isolation.	Relational SQLite Schema & Triggers: Eleven interlinked database tables with cascade rules.	Guarantees transactional consistency for wallet values, ticket histories, and audit records.
Chapter Implementation	5: Verification of payment systems and automated ledger postings.	Xendit Webhook Proxy & Approval Panel: Backend processing of payment gateway responses.	Implements double-entry validation preventing balance manipulation and unauthorized tier escalations.
Chapter 5: Testing	Automated code evaluation, schema integrity testing, and error handling.	Test Suite (dbTest.ts & systemTest.ts): Automated test scripts validating core API layers.	Verifies correct application performance under edge cases before system deployment.
Chapter 6: Evaluation	Performance reporting, organic social reach values, and ROI assessment.	Insights Analytics Charting: Recharts-based visual graphs depicting digital growth.	Empirically proves utility to users by translating raw analytics into local currency ROI equivalents.

4. System Architecture

Kawayan AI utilizes a hybrid application design. Below is the system-wide architecture diagram illustrating the interaction routes:

```
graph TD
  subgraph Presentation [Presentation Layer - Browser client]
    ReactUI[React SPA Components]
    UDS_Client[Universal Database Service: Client Mode]
    SocketIO_Client[Socket.io WebRTC Signaler]
  end

  subgraph Gateway [Gateway Layer - Node.js Express server]
    Server[server.js Express API]
    UDS_Server[Universal Database Service: Server Mode]
    JWT[JWT Authentication Middleware]
    Multer[Multer File Uploader]
    SocketIO_Server[Socket.io Signaling Server]
  end

  subgraph Data [Data Layer]
    SQLite[(SQLite DB: kawayan.db)]
    BrowserStorage[(Browser localStorage)]
  end

  subgraph Intelligent [Intelligent Services]
    Gemini[Google Gemini API]
    Unsloth[Unsloth OpenAI-Compatible Proxy]
    Ollama[Ollama Local Engine: Port 11434]
  end
```

```

subgraph Financial [Financial & Third-Party APIs]
  Xendit[Xendit Payment Gateway]
  Scrapers[Public Social Scrapers]
  Pollinations[Pollinations AI Image Generator]
end

%% Communication channels
ReactUI <--> |HTTP Requests + JWT Bearer| Server
ReactUI <--> |WebSockets / Call Signaling| SocketIO_Server
UDS_Client <--> |Read/Write| BrowserStorage

Server --> |Uses| JWT
Server --> |Uses| Multer
Server <--> |Queries| SQLite

Server <--> |HTTPS| Gemini
Server <--> |HTTPS Proxy| Unsloth
Server <--> |HTTP Loopback| Ollama

Server <--> |HTTPS Basic Auth| Xendit
Server <--> |HTTP GET| Scrapers
ReactUI --> |Direct HTTPS| Pollinations

```

Component Communication Protocol

1. **Client-to-Server REST Communication:** The React application sends asynchronous JSON payloads via fetch calls. All authenticated routes require a `Authorization: Bearer <JWT_TOKEN>` header.
2. **Real-time WebRTC Signaling:** When an SME starts a call, a WebSocket connection is initialized via `Socket.io`. The server routes SDP offers, answers, and ICE candidates between the SME client and the active Support Agent.
3. **Database Operations Layering:** Database commands are routed through `UniversalDatabaseService.ts`. In Node environments (during server-side seed, migrations, or CLI tests), it initializes `DatabaseService.ts` to access SQLite. In browser sessions, it uses `ClientDatabaseService.ts` to make REST requests to the backend.
4. **AI Generation Handshake:**
5. The frontend requests content generation.
6. The backend checks configuration variables.
7. If `UNSLOTH_API_URL` is set, it proxies requests to the custom Unsloth completions engine.
8. If Unsloth/Gemini Cloud queries fail, the backend attempts to fallback to a loopback address `http://127.0.0.1:11434` for Ollama.

5. Complete System Flow

Below is the workflow sequence showing user registration, profile definition, content generation, wallet checkout, WebRTC support call, and system audit logging:

```

sequenceDiagram
    autonumber
    actor User as SME User
    participant Web as React Web UI
    participant Server as Express API Server
    participant DB as SQLite Database
    participant Gemini as Gemini/Unsloth AI
    participant Xendit as Xendit Payments
    participant Peer as Support Agent

    %% Registration & Profile
    User->>Web: Register Account & Input Business Name
    Web->>Server: POST /api/auth/register
    Server->>DB: INSERT INTO users & wallets (FREE)
    DB-->>Server: Success
    Server-->>Web: Return User JWT Token
    User->>Web: Fill Brand Survey Profile
    Web->>Server: POST /api/profiles
    Server->>DB: INSERT/REPLACE INTO brand_profiles
    DB-->>Server: Success

    %% Content Generation Flow
    User->>Web: Click "Generate Content Plan"
    Web->>Server: POST /api/plans (Retrieve trends & generate ideas)
    Server->>Gemini: Fetch local Taglish topics based on profile
    Gemini-->>Server: Return content calendar (JSON)
    Server->>DB: INSERT INTO content_plans
    Server-->>Web: Display content grid

    %% Purchase & Billing Flow
    User->>Web: Choose Upgrade to Pro (PHP 500)
    Web->>Server: POST /api/wallet/xendit-checkout
    Server->>Xendit: POST /v2/invoices (GCash/Maya)

```

```

Xendit-->>Server: Return Checkout URL
Server-->>DB: INSERT INTO transactions (PENDING)
Server-->>Web: Open Checkout Modal / Redirect
User-->>Xendit: Completes Payment on GCash/Maya Portal
Xendit-->>Server: POST /api/webhooks/xendit (Payment Received)
Server-->>DB: UPDATE transactions (COMPLETED) & wallets (PRO)
Server-->>Web: Refresh wallet balance

%% Support WebRTC Flow
User-->>Web: Click "Call Support"
Web-->>Server: POST /api/support/calls/register
Server-->>DB: INSERT INTO active_calls
Server-->>Peer: Broadcast Call Waiting via Socket.io
Peer-->>Web: Accept Call Queue
Peer-->>Web: Open WebRTC SDP Handshake (Socket.io Signaling)
Web-->>User: Audio / Video Connection Active
User-->>Peer: Explains query
User-->>Web: Hang Up Call
Web-->>Server: POST /api/support/calls/unregister
Server-->>DB: INSERT INTO call_history & DELETE active_calls
Server-->>DB: INSERT INTO audit_logs (User finished call)

```

6. Module-by-Module Explanation

A. Authentication & User Management Module

- **Purpose:** Manages user registration, secure login, password modifications, and session checks.
- **Responsibilities:** Password hashing, JWT creation/verification, role checks, and database updates.
- **Inputs:** Email, Password, Role (User, Support, Admin), Business Name.
- **Outputs:** User details object, JSON Web Token (JWT).
- **Database Tables:** users, sessions.
- **API Endpoints:**
 - POST /api/auth/register: Evaluates password requirements, hashes the password, inserts record, returns JWT.
 - POST /api/auth/login: Checks matching credentials, issues JWT token.
 - POST /api/auth/logout: Revokes active sessions.
 - GET /api/auth/me: Authenticates active tokens and returns user identity.
- **Validation & Error Handling:** Email uniqueness, password criteria (length, caps, special character). Returns 401 Unauthorized for bad credentials, 409 Conflict for duplicate emails, and 400 Bad Request for password criteria failures.
- **Permissions:** Public access for Register/Login. Setting preferences/passwords restricted to matching owner or Admin role.

B. Content Calendar & Planning Module

- **Purpose:** Handles automated calendar schedules, post generation, and asset editing.
- **Responsibilities:** Normalizes ideas count to tier limitations, queries AI engines, generates visual prompts, and updates draft states.
- **Inputs:** Target month, brand profile context, single post edit text.
- **Outputs:** Array of generated post calendars, generated captions, visual prompts.
- **Database Tables:** content_plans, generated_posts.
- **API Endpoints:**
 - POST /api/plans: Save/generate ideas.
 - GET /api/plans/:userId/:month: Retrieve saved calendar plans.
 - POST /api/posts: Save content cards. Checks tier limits before processing.
 - GET /api/posts/user/:userId: Fetch generated calendar items.
- **Validation & Error Handling:** Limits generation rates based on subscriber tiers (utils/tierLimits.ts). Limits regeneration counts to a maximum of 2. If Gemini Cloud API fails, triggers local Ollama backup.
- **Permissions:** Users can write/update own posts. Admin/Support have read access.

C. Wallet & Invoicing Module

- **Purpose:** Coordinates prepaid top-ups, invoices, subscriptions, and transaction logs.
- **Responsibilities:** Processes payments, updates balances, handles webhooks, and verifies transactions.
- **Inputs:** Recharge amount, payment method (GCash, Maya, Card), transaction references.
- **Outputs:** Xendit checkout URL, transaction structures, updated balance objects.
- **Database Tables:** wallets, transactions.

- **API Endpoints:**
- GET /api/wallet/:userId: Retrieve active balance and subscription tier.
- POST /api/wallet/xendit-checkout: Initiates Xendit transaction logs. Updates balances directly if simulation modes are active.
- POST /api/wallet/verify-payment: Checks transactions status with Xendit API.
- POST /api/webhooks/xendit: Webhook endpoint processing signed transaction callbacks from Xendit.
- **Validation & Error Handling:** Signature verification on incoming webhooks. Basic Auth header validation on requests to external APIs. If billing accounts contain insufficient funds, debit operations fail with structured error responses.
- **Permissions:** Users can make purchases for their own accounts. Support can view billing records. Admins can manually adjust balances and adjust subscriptions.

D. Customer Support & VoIP Module

- **Purpose:** Real-time help desk ticketing and VoIP connection signaling.
- **Responsibilities:** Sequential ticket ID tracking, message storage, live queue management, and WebRTC signalling.
- **Inputs:** Ticket subjects, priority levels, room names, SDP payloads.
- **Outputs:** Ticket records, active queues, message strings, WebRTC connection states.
- **Database Tables:** tickets, active_calls, call_history.
- **API Endpoints:**
- POST /api/support/tickets: Creates tickets. Ensures sequential ticket numbers starting from 1001.
- PUT /api/support/tickets/:id: Appends responses or updates status.
- GET /api/support/calls: Lists queue requests.
- POST /api/support/calls/register: Adds call to queue.
- POST /api/support/calls/unregister: Resolves active call connection.
- **Validation & Error Handling:** Validates ticket priorities and categories (Technical, Billing, General).
- **Permissions:** Users can create/update their own tickets. Support and Admin roles can view, resolve, and initiate support calls.

7. Database Documentation

This section defines the data structures within kawayan.db.

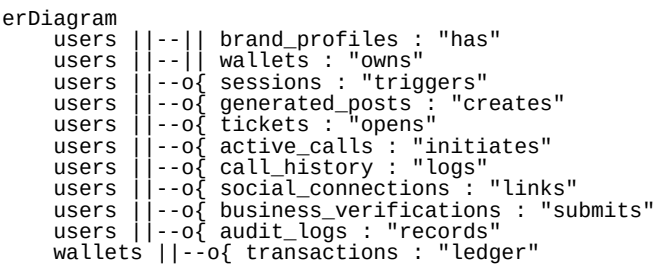


Table Definitions

1. users

Stores login credentials, system roles, and UI display settings. * **Primary Key:** id (TEXT) * **Columns:** * id (TEXT, PK): Unique user identifier. * email (TEXT, UNIQUE, NOT NULL): Account email. * password_hash (TEXT, NOT NULL): Hashed password. * role (TEXT, CHECK role IN ('user', 'admin', 'support'), NOT NULL): System access permissions. * business_name (TEXT, NULL): Display name. * theme (TEXT, DEFAULT 'light'): UI theme. * created_at (DATETIME, DEFAULT CURRENT_TIMESTAMP) * updated_at (DATETIME, DEFAULT CURRENT_TIMESTAMP)

2. brand_profiles

Stores brand identity metadata. Uses ON DELETE CASCADE linked to user ID. * **Primary Key:** id (TEXT) * **Foreign Key:** user_id (TEXT references users(id)) * **Columns:** * id (TEXT, PK) * user_id (TEXT, NOT NULL) * business_name (TEXT, NOT NULL) * industry (TEXT, NOT NULL) * target_audience (TEXT, NOT NULL) * brand_voice (TEXT, NOT NULL) * key_themes (TEXT, NOT NULL): Core themes. * brand_colors (TEXT, NULL): JSON Hex array. * contact_email (TEXT, NULL) * contact_phone (TEXT, NULL) * created_at (DATETIME, DEFAULT CURRENT_TIMESTAMP) * updated_at (DATETIME, DEFAULT CURRENT_TIMESTAMP)

3. generated_posts

Contains AI-generated content records. * **Primary Key:** id (TEXT) * **Foreign Key:** user_id (TEXT references users(id)) * **Columns:** * id (TEXT, PK) * user_id (TEXT, NOT NULL) * date (TEXT, NOT NULL): Scheduled publication date. * topic (TEXT, NOT NULL) * caption (TEXT, NOT NULL): Generated Taglish caption. * image_prompt (TEXT, NOT NULL) * image_url (TEXT, NULL): Asset data. * status (TEXT, CHECK status IN ('Draft', 'Scheduled', 'Published'), NOT NULL) * virality_score (INTEGER, CHECK 0-100) * virality_reason (TEXT, NULL) * format (TEXT, NULL) * external_link (TEXT, NULL) * published_at (DATETIME, NULL) * regen_count (INTEGER, DEFAULT 0) * history (TEXT, NULL): JSON Array of previous drafts.

4. wallets

Tracks current prepaid balance and active plan details. * **Primary Key:** user_id (TEXT references users(id)) * **Columns:** * user_id (TEXT, PK) * balance (REAL, DEFAULT 0.0) * currency (TEXT, DEFAULT 'PHP') * subscription (TEXT, CHECK subscription IN ('FREE', 'PRO', 'ENTERPRISE'), DEFAULT 'FREE') * updated_at (DATETIME, DEFAULT CURRENT_TIMESTAMP)

5. transactions

Double-entry transaction ledger logs. * **Primary Key:** id (TEXT) * **Foreign Key:** user_id (TEXT references wallets(user_id)) * **Columns:** * id (TEXT, PK) * user_id (TEXT, NOT NULL) * date (DATETIME, DEFAULT CURRENT_TIMESTAMP) * description (TEXT, NOT NULL) * amount (REAL, NOT NULL) * status (TEXT, CHECK status IN ('PENDING', 'COMPLETED', 'FAILED', 'CANCELLED'), NOT NULL) * type (TEXT, CHECK type IN ('CREDIT', 'DEBIT'), NOT NULL)

6. tickets

Central ticketing repository. Sequential ticket IDs begin at 1001. * **Primary Key:** id (TEXT) * **Foreign Key:** user_id (TEXT references users(id)) * **Columns:** * id (TEXT, PK) * ticket_num (INTEGER, NOT NULL): Sequential ticket ID. * user_id (TEXT, NOT NULL) * user_email (TEXT, NOT NULL) * subject (TEXT, NOT NULL) * priority (TEXT, CHECK priority IN ('Low', 'Medium', 'High', 'Critical'), NOT NULL) * status (TEXT, CHECK status IN ('Open', 'Pending', 'Resolved'), NOT NULL) * category (TEXT, NOT NULL, DEFAULT 'General') * messages (TEXT, NULL): JSON message history array. * created_at (DATETIME, DEFAULT CURRENT_TIMESTAMP) * updated_at (DATETIME, DEFAULT CURRENT_TIMESTAMP)

7. active_calls

Live WebRTC call requests queue. * **Primary Key:** user_id (TEXT references users(id)) * **Columns:** * user_id (TEXT, PK) * user_email (TEXT, NOT NULL) * room_name (TEXT, NOT NULL) * reason (TEXT, NULL) * started_at (DATETIME, DEFAULT CURRENT_TIMESTAMP)

8. call_history

Completed VoIP calls logs. * **Primary Key:** id (TEXT) * **Foreign Key:** user_id (TEXT references users(id)) * **Columns:** * id (TEXT, PK) * user_id (TEXT, NOT NULL) * user_email (TEXT, NOT NULL) * call_id (TEXT, NULL) * reason (TEXT, NULL) * started_at (DATETIME, NOT NULL) * ended_at (DATETIME, DEFAULT CURRENT_TIMESTAMP) * duration_seconds (INTEGER, NULL) * agent_id (TEXT, NULL)

9. social_connections

Linked social media credentials and statistics cache. * **Primary Key:** id (TEXT) * **Foreign Key:** user_id (TEXT references users(id)) * **Columns:** * id (TEXT, PK) * user_id (TEXT, NOT NULL) * platform (TEXT, CHECK platform IN ('facebook', 'instagram', 'tiktok'), NOT NULL) * connected (INTEGER, DEFAULT 0) * username (TEXT, NULL) * access_token (TEXT, NULL) * followers (INTEGER, DEFAULT 0) * engagement (REAL, DEFAULT 0.0) * data (TEXT, NULL): JSON platform data. * created_at (DATETIME, DEFAULT CURRENT_TIMESTAMP) * updated_at (DATETIME, DEFAULT CURRENT_TIMESTAMP)

10. business_verifications

Regulatory verification documents. * **Primary Key:** id (TEXT) * **Foreign Key:** user_id (TEXT references users(id)) * **Columns:** * id (TEXT, PK) * user_id (TEXT, NOT NULL, UNIQUE) * business_address (TEXT, NOT NULL) * business_phone (TEXT, NOT NULL) * document_name (TEXT, NOT NULL) * document_path (TEXT, NOT NULL): Multer upload path. * status (TEXT, CHECK status IN ('pending', 'verified', 'rejected'), DEFAULT 'pending') * rejection_reason (TEXT, NULL) * reviewed_by (TEXT, NULL) * reviewed_at (DATETIME, NULL) * created_at (DATETIME, DEFAULT CURRENT_TIMESTAMP) * updated_at (DATETIME, DEFAULT CURRENT_TIMESTAMP)

11. audit_logs

Security and administrative audit trail logs. * **Primary Key:** id (TEXT) * **Foreign Key:** user_id (TEXT

references users(id)) * **Columns:** * id (TEXT, PK) * user_id (TEXT, NOT NULL) * action (TEXT, NOT NULL) * details (TEXT, NULL): Metadata detail string. * timestamp (DATETIME, DEFAULT CURRENT_TIMESTAMP)

8. Folder Structure

The Kawayan AI directory structure is organized as follows:

```
/workspaces/Kawayan
|-- server.js                # Main Express Backend Entry Point & API Controller
|-- package.json            # Script Runner & Node Dependency Manifest
|-- tsconfig.json           # TypeScript Compiler Rules
|-- vite.config.ts          # Vite Configuration & Dev Proxy Definitions
|-- dev.sh                  # Setup Script (Ollama serv + launch dev:full)
|-- seed.ts                 # Database Seeder Data Definitions
|-- seed.sh                 # Script to purge and seed Database
|-- index.html              # Single Page Application HTML shell
|-- App.tsx                 # Core React Router & View State Coordinator
|-- index.tsx               # Client bootstrap hook
|-- components/             # React UI Components
|   |-- calendar/           # Calendar specific views (Schedule-X)
|   |-- AdminDashboard.tsx  # Admin administration panel
|   |-- SupportDashboard.tsx # Support Agent queue workspace
|   |-- SupportWidget.tsx   # Floating user-facing help ticket & call panel
|   |-- ContentCalendar.tsx # Core SME user workspace view
|   |-- InsightsDashboard.tsx # Social media growth and metrics console
|   |-- Billing.tsx         # Prepaid billing log & subscription panel
|   |-- BrandSurvey.tsx    # Profile setup wizard
|   |-- CallOverlay.tsx    # Live VoIP call WebRTC client interface
|   |-- LandingPage.tsx    # Marketing homepage
|   |-- Login.tsx          # Login interface
|   |-- XenditCheckoutModal.x # Local simulation checkout screen
|   |-- VerificationStatus.tx # Verification upload portal
|-- services/               # Service Integration Layer (Decoupling UI from Data)
|   |-- universalDatabaseService.ts # Environment detector switching SQLite/Client
|   |-- databaseService.ts      # SQLite Query execution methods (better-sqlite3)
|   |-- clientDatabaseService.ts # Frontend API caller fetching Express REST
|   |-- geminiService.ts        # Integrates with Google GenAI SDK & trends RSS
|   |-- jwtService.ts           # Implements session token operations & validation
|   |-- paymentService.ts       # Frontend proxy to process checkouts
|   |-- socialService.ts        # Redirect redirects & handles scraping callbacks
|   |-- validationService.ts    # Validates user profiles & passwords (Zod schemes)
|-- utils/                  # Shared helper utilities
|   |-- logger.ts              # Logs standard execution metrics and errors
|   |-- tierLimits.ts           # Tracks subscription limitation constraints
|-- tests/                  # Automation test configurations
|   |-- dbTest.ts              # Direct schema validation test
|   |-- systemTest.ts          # Broad database & logger tests
|-- extension/              # Browser extension integration sources
```

9. Code Flow

Below is the request-response lifecycle for database operations. It illustrates how components utilize the Universal Database Wrapper to query database engines depending on the execution context.

```
graph LR
    subgraph ClientSide [React Client Browser]
        UI[React UI Component]
        UDS_C[Universal Database Service]
        ClientService[ClientDatabaseService]
    end

    subgraph BackendSide [Express Backend Server]
        Route[server.js REST Endpoint]
        Middleware[JWT Auth Middleware]
        UDS_S[DatabaseService Node]
        SQLite[(SQLite DB)]
    end

    UI --> |Call Method| UDS_C
    UDS_C --> |Identify Client Environment| ClientService
    ClientService --> |fetch /api/auth/me + JWT header| Route
    Route --> |Intercept & Verify| Middleware
    Middleware --> |Token OK| UDS_S
    UDS_S --> |better-sqlite3 SQL execute| SQLite
    SQLite --> |Record Set| UDS_S
    UDS_S --> |Return JSON| Route
    Route --> |Response status 200| ClientService
    ClientService --> |Resolve Promise| UDS_C
    UDS_C --> |Update Component State| UI
```

Request Lifecycle Execution Flow

1. **User Action:** The user edits a draft caption on the interactive editorial calendar.
 2. **Component Hook:** The component calls `UniversalDatabaseService.savePost(post)`.
 3. **Client-Side Service Execution:** Since the code is running in the browser client context:
 4. `UniversalDatabaseService` forwards execution to the client-specific module `ClientDatabaseService.ts`.
 5. `ClientDatabaseService` makes a fetch request: `POST /api/posts`.
 6. The Authorization Header is appended: `Authorization: Bearer <jwt_token>`.
 7. **Backend Gateway Interception:**
 8. The Node Express API in `server.js` catches the request.
 9. The `authenticateToken` middleware executes.
 10. The token is verified using `JWTService.verifyToken()`. If valid, the user identity object is attached to `req.user`.
 11. **Database Transaction Execution:**
 12. The route handler forwards the request payload to `DatabaseService.savePost(post)`.
 13. `DatabaseService` checks constraints like `assertTierAllowsNewPost` to prevent tier limit breaches.
 14. `DatabaseService` runs a prepared statement against the database using `better-sqlite3`: `sql INSERT OR REPLACE INTO generated_posts (id, user_id, date, topic, caption, image_prompt, status) ...`
 15. **Unified Database Response Handshake:**
 16. The database returns the command results.
 17. `server.js` sends a 200 OK response containing a success JSON message.
 18. `ClientDatabaseService` resolves the Promise.
 19. The React UI displays a success toast.
-

10. User Guide

1. Registering & Compliance

1. Open the landing page and click **Get Started**.
2. Fill out the registration form. Select your role as **User** and enter your business name.
3. Upon registration, you will be prompted to submit compliance verification documents:
4. Enter your business address and phone number.
5. Upload a JPEG/PDF copy of your DTI/SEC registration or business permit.
6. Click **Submit**. Your verification status will show as **Pending** while administrators review it.

2. Setting Up Your Brand Profile

1. On your first login, complete the Brand Survey:
2. **Industry:** Enter your industry type (e.g., F&B, Retail).
3. **Target Audience:** Define your target demographic.
4. **Brand Voice:** Select a brand voice (e.g., Professional, Friendly).
5. **Key Themes:** Enter comma-separated themes.
6. Click **Save Brand Profile**. This profile provides context for the AI when generating Taglish captions.

3. Using the Editorial Calendar

1. The **Calendar** dashboard displays the monthly view.
2. Click **Generate Content Plan** at the top right to fetch current trending topics and generate a localized calendar plan.
3. Click any content slot inside the grid to open the editing pane:
4. Review the generated Taglish caption and visual prompt.
5. **Regenerate:** If you want another draft, click **Regenerate Caption** (up to 2 times).
6. **Custom Captions:** You can manually edit the caption text.
7. **Visual Assets:** Click **Upload Image** to add custom media, or let the system render an image using the visual prompt.
8. Click **Save Draft** to store changes, or set status to **Scheduled** when ready to queue the post.

4. Billing & Purchases

1. Click **Billing** in the navigation bar to check your account ledger.
2. **Upgrades:** Click **Upgrade to Pro** (PHP 500/month) to increase your monthly post limit from 8 to 16.
3. **Wallet Top-up:** Enter a custom amount (e.g., PHP 150) and select GCash, Maya, or Card:
4. You will be redirected to the secure Xendit invoice page.
5. Enter your credentials on Xendit's secure portal.
6. Upon successful verification, your wallet balance will automatically update.
7. **Single Add-on Purchase:** If you reach your monthly tier limit, click any empty calendar slot and select **Purchase Add-on Post** (PHP 150) to buy a single slot.

5. Growth Insights

1. Select **Growth Insights** in the navigation bar.
2. Connect your accounts to view followers, views, and engagement rates for TikTok, Facebook, and Instagram.
3. The dashboard displays growth trends over time and estimates your Digital ROI based on your organic social reach.

6. Support Ticketing & VoIP Calls

1. Click the floating **Support Bubble** in the bottom-right corner.
2. **Ticketing:** Select **New Ticket**, enter a subject, category (Technical, Billing, General), and message, then click **Submit**.
3. **Support Call:** If you need real-time assistance, click **Call Support**:
4. Grant microphone and camera permissions.
5. Wait in the queue for a support agent to pick up your call.
6. Use the controls to mute or toggle your camera.

11. Administrator Guide

Administrators can access the administrative dashboard at `admin@kawayan.ph` with password `Admin123!`.

Management Tools & Controls

1. **Analytics Overview:** View revenue growth, signup rates, monthly retention, and transaction logs.
2. **User Administration:**
3. **Edit Users:** View, edit, or delete accounts.
4. **Adjust Subscriptions:** Upgrade users to PRO or ENTERPRISE and adjust billing cycles.
5. **Adjust Wallet Balances:** Manually credit or debit account ledger balances.
6. **Payment Approvals:** View pending transactions. Approve transactions manually if automatic Xendit updates fail.
7. **Verification Portal:**
8. View submitted documents (DTI/SEC/Business Permit).
9. Click **Approve** to verify accounts, or **Reject** and provide a reason for resubmission.
10. **Security Log Auditing:** View the full list of system audit logs, tracking logins, wallet updates, and support call connections.

12. Security Features

- **Password Hashing:** Passwords are encrypted before database insertion using `bcryptjs` with a cost factor of 10.
- **Session Management:** Implements stateless JWT tokens (`jsonwebtoken`). Tokens are stored in browser `localStorage`.
- **Role-Based Access Control:** Middleware verifies user roles (`admin` or `support`) before granting access to sensitive routes: `js const requireAdmin = (req, res, next) => { if (!req.user || req.user.role !== 'admin') { return res.status(403).json({ error: 'Admin access required' }); } next(); };`
- **SQL Injection Mitigation:** Prepared statements and parameterized inputs using the `better-sqlite3` library prevent SQL injection: `js db.prepare('SELECT * FROM users WHERE email = ?').get(email);`
- **Cross-Site Scripting (XSS) Prevention:** Vite and React automatically escape values rendered in JSX templates. The server also sends strict Content Security Policy (CSP) headers restricting script execution sources.
- **File Upload Security:** Uploaded compliance documents are whitelisted by file type (JPG, PNG, PDF)

and size (under 5 MB) via Multer configurations.

13. Technologies Used

Technology Component	Functional Role	Reason for Selection	Key Advantages
React (TypeScript)	Presentation Layer	High component reusability and strong typing.	Catches compile-time errors, improving system stability.
Vite	Frontend Build Engine	Fast compilation times and efficient hot reloading.	Speeds up development iterations.
Express.js (Node.js)	Backend Router Gateway	Lightweight event-driven model.	Efficiently manages multiple requests and proxies.
better-sqlite3	Persistence Layer	Fast SQLite library for Node.js.	High execution speeds without external service dependencies.
Socket.io	Real-time signaling	Cross-browser WebSocket layer.	Fallbacks prevent client disconnected states.
Xendit API	Payment Integrations	Seamless regional payment support.	Simple API integration for local payment systems.
Google GenAI SDK	Cloud AI Generation	Access to Google's cloud AI infrastructure.	High-quality Taglish text generation.
simple-peer	WebRTC abstraction	Simplified WebRTC library.	Simplifies peer-to-peer connection setup for support calls.

14. Algorithms and Business Logic

1. Hybrid AI Fallback Algorithm

The system manages fallbacks between cloud Gemini models and local Ollama instances to ensure availability:

```
graph TD
  A[Start Content Generation] --> B{Is Cloud AI responsive?}
  B -->|Yes| C[Generate calendar via Gemini API]
  B -->|No| D[Log connection failure alert]
  D --> E{Is local Ollama server running?}
  E -->|Yes| F[Fallback to qwen2.5:7b via loopback]
  E -->|No| G[Return last cached content plan template]
```

2. Tier Limits Enforcement

The backend checks subscriber limits (TRIAL_POST_LIMIT vs PRO_POST_LIMIT) before saving posts. Purchases with an addon- prefix bypass these limits:

```
export async function assertTierAllowsNewPost(userId: string, postId: string) {
  if (postId.startsWith('addon-')) return; // Bypasses limits
  const wallet = await getWallet(userId);
  const currentPostCount = await countPostsInMonth(userId);
  const limit = getBatchLimitForSubscription(wallet.subscription);
  if (currentPostCount >= limit) {
    throw { code: 'TIER_LIMIT_REACHED', limit };
  }
}
```

3. ROI Metric Computation

ROI is calculated based on organic social reach, multiplying engagement metrics by a baseline unit value of PHP 0.05:
$$\text{Digital Value (PHP)} = (\text{Views} + \text{Interactions} + \text{Likes} + \text{Followers}) \times 0.05$$
$$\text{ROI} = \left(\frac{\text{Digital Value} - \text{Wallet Spend}}{\text{Wallet Spend}} \right) \times 100$$

15. Error Handling

- **Database Level:** Operations are wrapped in try-catch blocks. Database errors are logged with details like query and parameters while returning user-friendly messages.
 - **API Level:** Uncaught exceptions are handled by Express error-handling middleware, logging errors and returning standard error responses.
 - **Network Level:** The application checks connection status before operations, disabling features like AI generation when offline.
 - **Logger Utility:** Saves standard executions, warnings, database errors, and system events to the log file.
-

16. Testing

- **Functional Testing:** Checks page layouts, route navigation, forms validation, and calendar actions.
 - **Unit Testing:** Automated tests verify schema creations, validation services, and password hash routines.
 - **Integration Testing:** Verifies API endpoints and database operations using mock data payloads.
 - **User Acceptance Testing (UAT):** Simulated user roles test standard SME workflows, support ticket management, and administrator controls.
-

17. Future Improvements

1. **Direct Publishing:** Integrate official social media APIs (TikTok Content Posting API, Meta Graph API) to support direct scheduling and publishing.
 2. **Expanded Analytics:** Track historical user behavior trends and implement cohort analysis to calculate retention rates.
 3. **AI Media Generation:** Integrate image generation models directly into the backend to allow users to generate custom brand images.
 4. **Cloud Database Migration:** Support PostgreSQL in production for better scalability and concurrent connection handling.
-

18. Frequently Asked Questions (FAQ)

Q: How does the system handle database locking errors?

A: SQLite is configured in WAL (Write-Ahead Logging) mode, allowing concurrent readers without database locking issues.

Q: Can I run this system offline?

A: Yes. The system can run offline by using browser localStorage for frontend storage and a local Ollama instance for AI generation.

Q: Why are my payments not appearing instantly in the wallet?

A: Ensure your Xendit callback webhooks are properly configured. If they fail, an administrator can manually verify and approve the transaction.

19. Glossary

- **Taglish:** A blend of Tagalog and English commonly used in everyday Philippine communications.
- **SME:** Small and Medium Enterprises.

- **WebRTC**: Web Real-Time Communication. Allows peer-to-peer audio and video streaming in browser applications.
 - **WAL**: Write-Ahead Logging. A SQLite configuration that improves performance and concurrent read/write operations.
 - **JWT**: JSON Web Token. A secure, URL-safe method for representing claims between two parties.
 - **CORS**: Cross-Origin Resource Sharing. Security mechanism that restricts resource requests from external origins.
-

20. Conclusion

Kawayan AI implements localized AI social media calendar scheduling, WebRTC customer support, and Xendit billing integrations. The system matches the objectives of the Capstone project, offering Philippine SMEs an accessible and reliable tool to manage their digital brand presence.